

Solutions — Lab 1: Modeling and Identification of the Parrot AR.Drone

Questions 2.1 – 2.9

Setup and assumptions

We use the notation of the handout: $\mathbf{f}_T = [f_1 \ f_2 \ f_3 \ f_4]^T$ for the individual rotor thrusts, $n_i = cf_i$ for the individual reaction torques, r for the arm length (distance from the center of mass to each rotor), and $\mathbf{f}_g = mg\mathbf{e}_3$ for gravity expressed in $\{I\}$.

2.1 – Expressions for $\mathbf{f}$ and $\mathbf{n}$

Approval:	☐ Student approved	☐ Teaching staff approved
------------------	---	--

Translational force. The total external force in $\{B\}$ is the sum of gravity (expressed in $\{I\}$, rotated into $\{B\}$) and the resultant of the four rotor thrusts (already aligned with $\{B\}$, all along $-z_B$):

$$\mathbf{f} = R^T(\lambda) \mathbf{f}_g + N \mathbf{f}_T.$$

Hence

$$M = R^T(\lambda) \in SO(3), \quad N = -\mathbf{e}_3 \begin{bmatrix} 1 & 1 & 1 & 1 \end{bmatrix} = \begin{bmatrix} 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ -1 & -1 & -1 & -1 \end{bmatrix} \in \mathbb{R}^{3 \times 4}.$$

(M depends on attitude because gravity is defined in $\{I\}$ and must be rotated into $\{B\}$; N is constant because every thrust is already expressed in $\{B\}$ along the same axis.)

Torque. Each rotor i produces (a) a moment from its thrust acting at moment arm r about the two horizontal axes, and (b) a reaction torque $n_i = cf_i$ about z_B , alternating in sign with the spin direction. With the geometric convention above ($\mathbf{r}_i \times (-f_i \mathbf{e}_3)$ for each rotor position $\mathbf{r}_i$):

$$n_x = r(f_2 - f_4), \quad n_y = r(f_1 - f_3), \quad n_z = c(f_1 - f_2 + f_3 - f_4),$$

so that

$$\mathbf{n} = P \mathbf{f}_T, \quad P = \begin{bmatrix} 0 & r & 0 & -r \\ r & 0 & -r & 0 \\ c & -c & c & -c \end{bmatrix} \in \mathbb{R}^{3 \times 4}.$$

2.2 – Redefining $\{B\}$ with x_B at 45°

Approval:	☐ Student approved	☐ Teaching staff approved
------------------	---	--

Rotating the body axes by 45° about z_B (new x'_B midway between the arms of rotors 1 and 2) does *not* change $\mathbf{f}$: every thrust still acts along $-z_B$ (unchanged, since the rotation is about z_B

itself), and $R^T(\lambda)$ is still just the attitude rotation expressed with respect to whichever axes are called $\{B\}$ — its functional form does not depend on this relabeling. Hence

M, N unchanged.

$\mathbf{n} = P\mathbf{f}_T$ *does* change, because the horizontal position of every rotor relative to the (now rotated) x_B, y_B axes changes. The new x'_B is midway between the old $+x_B$ (rotor 1's arm) and old $-y_B$ (rotor 2's arm), i.e. $x'_B = (x_B - y_B)/\sqrt{2}$, $y'_B = (x_B + y_B)/\sqrt{2}$. Expressing each rotor position in the new basis gives $\mathbf{r}'_1 = \frac{r\sqrt{2}}{2}(1, 1, 0)$, $\mathbf{r}'_2 = \frac{r\sqrt{2}}{2}(1, -1, 0)$, $\mathbf{r}'_3 = \frac{r\sqrt{2}}{2}(-1, -1, 0)$, $\mathbf{r}'_4 = \frac{r\sqrt{2}}{2}(-1, 1, 0)$ — each rotor sits at 45° from both axes, so each thrust now contributes to *both* n_x and n_y (instead of purely one, as before). Using $\mathbf{r} \times (-f\mathbf{e}_3) = (-fr_y, fr_x, 0)$ for each rotor:

$$n_x = \frac{r\sqrt{2}}{2}(f_2 + f_3 - f_1 - f_4), \quad n_y = \frac{r\sqrt{2}}{2}(f_1 + f_2 - f_3 - f_4),$$

while n_z is unchanged (yaw reaction torque depends only on spin direction, not on the axes' in-plane orientation):

$$n_z = c(f_1 - f_2 + f_3 - f_4).$$

So

$$P' = \begin{bmatrix} -\frac{r\sqrt{2}}{2} & \frac{r\sqrt{2}}{2} & \frac{r\sqrt{2}}{2} & -\frac{r\sqrt{2}}{2} \\ \frac{r\sqrt{2}}{2} & \frac{r\sqrt{2}}{2} & -\frac{r\sqrt{2}}{2} & -\frac{r\sqrt{2}}{2} \\ c & -c & c & -c \end{bmatrix} \neq P.$$

2.3 – Input transformation $\mathbf{u} = L\mathbf{f}_T$

Approval: ☐ Student approved ☐ Teaching staff approved

Stacking $T = \sum_i f_i$ (from 2.1, first row all ones) on top of $\mathbf{n} = P\mathbf{f}_T$ (from 2.1) gives

$$\mathbf{u} = \begin{bmatrix} T \\ \mathbf{n} \end{bmatrix} = L\mathbf{f}_T, \quad L = \begin{bmatrix} 1 & 1 & 1 & 1 \\ 0 & r & 0 & -r \\ r & 0 & -r & 0 \\ c & -c & c & -c \end{bmatrix}.$$

Usefulness. L is a constant, invertible matrix ($\det L \neq 0$ for $r, c \neq 0$), so the map $\mathbf{f}_T \leftrightarrow \mathbf{u}$ is good. Its point is that the rigid-body dynamics (1)–(3) only ever depend on $\mathbf{f}_T$ through exactly these four combinations: T enters the translational dynamics (scaled along z_B) and $\mathbf{n}$ enters the rotational dynamics directly. The individual f_i are physically coupled (each one affects both T and one or two components of $\mathbf{n}$); $\mathbf{u}$ decouples the problem into four independent virtual channels — one for altitude/thrust and three for the body-frame moments (roll, pitch, yaw). This lets the height and attitude controllers be designed directly in terms of T and $\mathbf{n}$ (as done throughout the rest of the guide), with the physical per-rotor commands recovered only at the very last step via L^{-1} — i.e., L is exactly the control-allocation / mixer stage of a real flight controller. The only caveat is that not every $\mathbf{u}$ is achievable, since $\mathbf{f}_T$ must remain nonnegative and bounded.

Inverse transformation. Solving $L\mathbf{f}_T = \mathbf{u}$ directly (summing/subtracting pairs of rows):

$$f_1 = \frac{T}{4} + \frac{n_y}{2r} + \frac{n_z}{4c}, \quad f_2 = \frac{T}{4} + \frac{n_x}{2r} - \frac{n_z}{4c},$$

$$f_3 = \frac{T}{4} - \frac{n_y}{2r} + \frac{n_z}{4c}, \quad f_4 = \frac{T}{4} - \frac{n_x}{2r} - \frac{n_z}{4c},$$

i.e. $\mathbf{f}_T = L^{-1}\mathbf{u}$ with

$$L^{-1} = \begin{bmatrix} 1/4 & 0 & 1/(2r) & 1/(4c) \\ 1/4 & 1/(2r) & 0 & -1/(4c) \\ 1/4 & 0 & -1/(2r) & 1/(4c) \\ 1/4 & -1/(2r) & 0 & -1/(4c) \end{bmatrix}.$$

Each rotor receives a quarter of the total thrust plus/minus the roll, pitch and yaw contributions needed to produce the requested $\mathbf{n}$ — the familiar quadrotor mixer formula.

2.4 – Nonlinear state-space dynamics $\dot{x} = g(x, u)$

Approval: ☐ Student approved ☐ Teaching staff approved

$$\mathbf{f} = mg R^T(\lambda)\mathbf{e}_3 - T\mathbf{e}_3.$$

Kinematics for ${}^B\dot{p}$. Differentiating ${}^Bp = R^T p$,

$${}^B\dot{p} = \dot{R}^T p + R^T \dot{p}.$$

By definition of the body angular velocity (slide 24): $\dot{R} = RS(\omega)$. Using $S(\cdot)^T = -S(\cdot)$ (suggestion),

$$\dot{R}^T = S(\omega)^T R^T = -S(\omega)R^T.$$

Substituting, and using $\dot{p} = Rv$ (given kinematics) so $R^T \dot{p} = R^T Rv = v$:

$${}^B\dot{p} = -S(\omega)R^T p + v = v - S(\omega) {}^Bp.$$

Assembling $g(x, u)$. Combining the kinematics ($\dot{p} = Rv$, $\dot{\lambda} = Q(\lambda)\omega$) with the dynamics (1) and the substitutions above:

$$\dot{x} = g(x, u) = \begin{bmatrix} {}^B\dot{p} \\ \dot{v} \\ \dot{\lambda} \\ \dot{\omega} \end{bmatrix} = \begin{bmatrix} v - S(\omega) {}^Bp \\ -S(\omega)v + g R^T(\lambda)\mathbf{e}_3 - (T/m)\mathbf{e}_3 \\ Q(\lambda)\omega \\ J^{-1}(u_{2:4} - S(\omega)J\omega) \end{bmatrix}.$$

Alternative state x_1 . For $x_1 = [p^T \ v^T \ \lambda^T \ \omega^T]^T$ the position is kept in the inertial frame, so the first block is just the linear kinematics $\dot{p} = R(\lambda)v$. The other three blocks do not involve Bp and are unchanged:

$$\dot{x}_1 = g_1(x_1, u) = \begin{bmatrix} R(\lambda)v \\ -S(\omega)v + g R^T(\lambda)\mathbf{e}_3 - (T/m)\mathbf{e}_3 \\ Q(\lambda)\omega \\ J^{-1}(u_{2:4} - S(\omega)J\omega) \end{bmatrix}.$$

The two models differ only in the first block: $R(\lambda)v$ in g_1 versus $v - S(\omega) {}^Bp$ in g .

2.5 – Equilibrium point for hover

Approval: ☐ Student approved ☐ Teaching staff approved

Pedro's proposal:

By definition, (x_0, u_0) is an equilibrium of $\dot{x} = g(x, u)$ if and only if $g(x_0, u_0) = 0$, so the four blocks derived in 2.4 must vanish simultaneously:

$$\begin{aligned} v_0 - S(\omega_0) {}^B p_0 &= 0, & -S(\omega_0)v_0 + gR^T(\lambda_0)\mathbf{e}_3 - \frac{T_0}{m}\mathbf{e}_3 &= 0, \\ Q(\lambda_0)\omega_0 &= 0, & n_0 - S(\omega_0)J\omega_0 &= 0. \end{aligned}$$

We solve these one at a time. Since $Q(\lambda_0)$ is invertible for any sensible hover attitude ($\theta_0 \neq \pm\pi/2$), the third equation forces $\omega_0 = 0$. Plugging this back into the fourth gives $n_0 = 0$: no net moment at equilibrium, as expected. The first equation then collapses to $v_0 = 0$.

The second equation is where the actual work happens. With $\omega_0 = 0$ it reduces to $gR^T(\lambda_0)\mathbf{e}_3 = (T_0/m)\mathbf{e}_3$. Now $R^T(\lambda)\mathbf{e}_3$ is the third row of $R(\lambda)$ read as a column, namely $[-\sin\theta, \sin\phi\cos\theta, \cos\phi\cos\theta]^T$, so componentwise

$$-g\sin\theta_0 = 0, \quad g\sin\phi_0\cos\theta_0 = 0, \quad g\cos\phi_0\cos\theta_0 = T_0/m.$$

Restricting to the physical branch $\theta_0 \in (-\pi/2, \pi/2)$, the first equation gives $\theta_0 = 0$, which reduces the second to $\sin\phi_0 = 0$ and hence $\phi_0 = 0$, and finally the third gives $T_0 = mg$. Note that ψ_0 never appears: a rotation about z_B leaves $\mathbf{e}_3$ fixed, so any yaw is admissible and $\lambda_0 = [0, 0, \psi_0]^T$ with ψ_0 free.

Collecting everything,

$$x_0 = \begin{bmatrix} {}^B p_0 \\ v_0 \\ \lambda_0 \\ \omega_0 \end{bmatrix} = \begin{bmatrix} R_z^T(\psi_0)p_0 \\ \mathbf{0}_3 \\ [0, 0, \psi_0]^T \\ \mathbf{0}_3 \end{bmatrix}, \quad u_0 = \begin{bmatrix} T_0 \\ n_0 \end{bmatrix} = \begin{bmatrix} mg \\ \mathbf{0}_3 \end{bmatrix},$$

using ${}^B p_0 = R^T(\lambda_0)p_0 = R_z^T(\psi_0)p_0$ since $R(\lambda_0)$ reduces to a pure yaw rotation.

Physically this is exactly what one would expect: at hover the drone has no linear or angular velocity, no net torque, and no roll or pitch, so the thrust vector points straight up and cancels gravity, $T_0 = mg$. Both the yaw and the position are free because gravity and the thrust direction (along z_B) are invariant under rotation about z_B — any heading, at any point in space, is a valid hover. Feeding u_0 back through the mixer from 2.3 gives $\mathbf{f}_{T0} = L^{-1}u_0 = \frac{mg}{4}\mathbf{1}_4$, i.e. the four rotors share the load equally, as they should for a symmetric quadrotor holding position without yawing.

2.6 – Linearized dynamics $\dot{\tilde{x}} = A\tilde{x} + B\tilde{u}$

Approval: ☐ Student approved ☐ Teaching staff approved

Pedro's proposal:

Linearization strategy. Substitute $x = x_0 + \tilde{x}$, $u = u_0 + \tilde{u}$ into each block of g and keep only terms that are zeroth order (which cancel, since $g(x_0, u_0) = 0$) or first order in $(\tilde{x}, \tilde{u})$; any product of two or more tilde quantities is second order and is dropped. Since $v_0 = \omega_0 = 0$ and $\tilde{p}_0 = 0$ (from 2.5), most quadratic/bilinear terms below vanish identically – this is exactly the Jacobian $\partial g/\partial x, \partial g/\partial u$ evaluated at (x_0, u_0) , just obtained by direct substitution instead of symbolic differentiation.

Assembling A and B (state order $\tilde{x} = [\tilde{p}; \tilde{v}; \tilde{\lambda}; \tilde{\omega}]$, input order $\tilde{u} = [\tilde{T}; \tilde{n}]$):

$$A = \begin{bmatrix} 0_3 & I_3 & 0_3 & 0_3 \\ 0_3 & 0_3 & A_{v\lambda} & 0_3 \\ 0_3 & 0_3 & 0_3 & I_3 \\ 0_3 & 0_3 & 0_3 & 0_3 \end{bmatrix}, \quad A_{v\lambda} = \begin{bmatrix} 0 & -g & 0 \\ g & 0 & 0 \\ 0 & 0 & 0 \end{bmatrix},$$

$$B = \begin{bmatrix} 0_{3 \times 1} & 0_{3 \times 3} \\ b_T & 0_{3 \times 3} \\ 0_{3 \times 1} & 0_{3 \times 3} \\ 0_{3 \times 1} & J^{-1} \end{bmatrix}, \quad b_T = \begin{bmatrix} 0 \\ 0 \\ -1/m \end{bmatrix}.$$

2.7 – Transfer functions from the linearized dynamics

Approval: ☐ Student approved ☐ Teaching staff approved

With J diagonal, each block of A, B from 2.6 decouples into three independent second-order chains, read off row by row.

Attitude channels. $\dot{\tilde{\lambda}} = \tilde{\omega}$ and $\dot{\tilde{\omega}} = J^{-1}\tilde{n}$ give, componentwise, $\ddot{\phi} = \tilde{n}_x/J_{xx}$, $\ddot{\theta} = \tilde{n}_y/J_{yy}$, $\ddot{\psi} = \tilde{n}_z/J_{zz}$. Hence

$$G_\phi(s) = \frac{1}{J_{xx}s^2}, \quad G_\theta(s) = \frac{1}{J_{yy}s^2}, \quad G_\psi(s) = \frac{1}{J_{zz}s^2}.$$

Vertical channel. $\dot{\tilde{z}} = \tilde{v}$ and $\dot{\tilde{v}} = -\tilde{T}/m$ (third rows of the $\dot{\tilde{p}} = \tilde{v}$ and $\dot{\tilde{v}} = A_{v\lambda}\tilde{\lambda} + b_T\tilde{T}$ blocks) give $\ddot{\tilde{z}} = -\tilde{T}/m$, so

$$G_z(s) = \frac{Z(s)}{T(s)} = -\frac{1}{ms^2}.$$

Horizontal channels. From $A_{v\lambda}$: $\dot{\tilde{u}} = -g\tilde{\theta}$ and $\dot{\tilde{v}} = g\tilde{\phi}$ (first two rows), together with $\dot{\tilde{x}} = \tilde{u}$, $\dot{\tilde{y}} = \tilde{v}$, give $\ddot{\tilde{x}} = -g\tilde{\theta}$ and $\ddot{\tilde{y}} = g\tilde{\phi}$. Substituting $\Theta(s) = G_\theta(s)N_y(s)$ and $\Phi(s) = G_\phi(s)N_x(s)$:

$$G_x(s) = \frac{X(s)}{N_y(s)} = -\frac{g}{J_{yy}s^4}, \quad G_y(s) = \frac{Y(s)}{N_x(s)} = \frac{g}{J_{xx}s^4}.$$

Note $\tilde{x}, \tilde{y}, \tilde{z}$ here are the components of ${}^B\tilde{p}$ (the state chosen in 2.4–2.6), not of the inertial p directly – relevant for the discussion in 2.8.

2.8 – Relating G_ϕ, G_y and G_θ, G_x

Approval: ☐ Student approved ☐ Teaching staff approved

Dividing the horizontal transfer functions from 2.7 by the corresponding attitude ones:

$$G_y(s) = \frac{g}{s^2} G_\phi(s), \quad G_x(s) = -\frac{g}{s^2} G_\theta(s).$$

Physical interpretation. Tilting about x_B (roll, ϕ) tips the thrust vector toward y_B , producing a horizontal acceleration $g\phi$; tilting about y_B (pitch, θ) tips it toward $-x_B$, producing $-g\theta$. Position is

acceleration integrated twice, so each horizontal transfer function is exactly its attitude counterpart cascaded with two extra integrators and a gain g : attitude must first build up before it can move the vehicle, it never happens instantaneously.

Effect of ψ_0 . $\tilde{x}, \tilde{y}$ are body-frame quantities (${}^B\tilde{p}$), so they relate to the actual inertial horizontal displacement via $R_z(\psi_0)$, which mixes x and y whenever $\psi_0 \neq 0$ (unlike $\tilde{z}$, which is unaffected by yaw, cf. 2.9). A roll input therefore does not move the vehicle purely along inertial North or East once $\psi_0 \neq 0$: the decoupled pairing $G_\phi \leftrightarrow G_y$, $G_\theta \leftrightarrow G_x$ only holds in body coordinates, and any outer position-loop working in the inertial frame must rotate its commanded accelerations by ψ_0 before splitting them into roll/pitch references – exactly the yaw-compensation stage present between the position and attitude loops of a real flight controller.

2.9 – Height control schemes

Approval: ☐ Student approved ☐ Teaching staff approved

Plant. With $h = -z$ and $h_0 = -z_0$, $\tilde{h} = -\tilde{z}$; since equilibrium hover has $\phi_0 = \theta_0 = 0$, yaw alone does not affect the z -component (cf. 2.8), so $\tilde{z} = {}^B\tilde{p}_z$ exactly and $H(s) = -\tilde{Z}(s)$. From 2.7, $G_z(s) = -1/(ms^2)$, so

$$\frac{H(s)}{T(s)} = -G_z(s) = \frac{1}{ms^2},$$

matching the cascade $\frac{1}{ms} \rightarrow \frac{1}{s}$ (force $\rightarrow \dot{h} \rightarrow h$) shown in Figs. 2–3.

Closed-loop transfer functions. For Fig. 3 (series PD), $T = (k_p + k_d s)(H_{ref} - H)$ directly gives

$$\frac{H(s)}{H_{ref}(s)} = \frac{k_d s + k_p}{ms^2 + k_d s + k_p}.$$

For Fig. 2 (cascade), the inner loop around $1/(ms)$ with feedback k_d gives $\dot{H} = \frac{1}{ms}[k_p(H_{ref} - H) - k_d s H]$; solving for H :

$$\frac{H(s)}{H_{ref}(s)} = \frac{k_p}{ms^2 + k_d s + k_p}.$$

The two share the same denominator (same poles, same root-locus in k_p for fixed k_d , cf. 2.10) but Fig. 3 has an extra zero at $s = -k_p/k_d$.

Differences. Writing $G_3 = G_2 + \frac{k_d s}{ms^2 + k_d s + k_p}$, the Fig. 3 step response equals the Fig. 2 response plus a term that starts at 0 with slope $k_d A/m > 0$: Fig. 3 always overshoots more for the same (k_p, k_d) . Computing $T(s)$ for a step $H_{ref}(s) = A/s$ in each scheme and applying the initial value theorem gives $T(0^+) = k_p A$ (finite) for Fig. 2, but $T(0^+) \rightarrow \infty$ for Fig. 3 – the series PD differentiates the reference itself, so a step h_{ref} demands an infinite instantaneous thrust (“derivative kick”), while the cascade never differentiates h_{ref} , only the measured h . Likewise, if $\dot{h}$ in Fig. 2 comes from a genuine rate measurement (rather than from numerically differentiating h), its noise-to-actuator gain is the flat $|k_d|$, whereas Fig. 3’s $k_d s$ branch has gain $|k_d \omega|$, unbounded in frequency – it amplifies high-frequency sensor noise that the cascade avoids.

Advantages/disadvantages. Fig. 2 (cascade): smoother, more predictable step response (no zero), no derivative kick, and – if a rate sensor is available – no derivative-induced noise amplification; but it needs access to a real $\dot{h}$ signal and is structurally more complex (nested loop). Fig. 3 (series PD): simpler to implement (single compensator) and reacts faster to reference changes, at the cost of more overshoot, an unrealizable derivative kick on step references, and noise amplification if $\dot{h}$ must be obtained by differentiating a noisy h .